

Introdução à Computação definindo funções em C

Alexandre Rademaker

funções I

C e quase todas as linguagens modernas nos permitem escrever funções, às vezes também conhecidas como procedimentos, métodos ou sub-rotinas. Funções permitem: organização, reuso e simplificação.

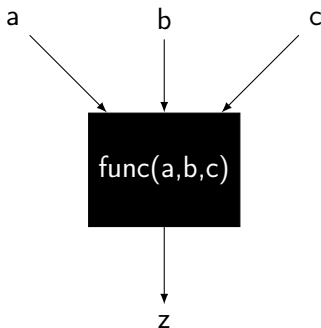
Modularização é importante para clareza e manutenção de código.

Podemos pensar em uma função como uma **caixa preta** que recebe zero ou mais entradas e produz uma saída.

Ao **usarmos** uma função, não precisamos saber detalhes de sua implementação.

funções II

O contrato de uma função é sua **assinatura**. O comportamento normalmente é estimado com base no nome. A maioria das funções tem nomes óbvios e são bem documentadas.



funções III

Funções podem ser declaradas antes da função `main`

```
return-type name(argument-list);
```

Exemplo:

```
int add_two_ints(int a, int b);
```

Esta função recebe duas entradas, os nomes são para serem referenciadas internamente na função. Nada especial sobre estes nomes, podem ser simples.

funções IV

Função podem ser definidas, seu comportamento especificado.
Como os **argumentos** devem ser usados para produzir a saída.

```
1 float mult_two_reals(float x, float y);  
2  
3 float mult_two_reals(float x, float y)  
4 {  
5     float product = x * y;  
6     return product;  
7 }
```

funções V

Agora, pense como podemos definir `add_two_ints`?

```
1 int add_two_ints(int x, int y);  
2  
3 int add_two_ints(int x, int y)  
4 {  
5  
6 }
```

funções VI

Agora, pense como podemos definir `add_two_ints`?

```
1 int add_two_ints(int x, int y)
2 {
3     int sum;
4     sum = x + y;
5     return sum;
6 }
```

funções VII

Agora, pense como podemos definir `add_two_ints`?

```
1 int add_two_ints(int x, int y)
2 {
3     int sum = x + y;
4     return sum;
5 }
```

funções VIII

Agora, pense como podemos definir `add_two_ints` ?

```
1 int add_two_ints(int a, int b)
2 {
3     int sum = 0;
4     if(a > 0)
5         for(int i = 0; i < a; sum++, i++);
6     else
7         for(int i = a; i < 0; sum--, i++);
8     if(b > 0)
9         for(int i = 0; i < b; sum++, i++);
10    else
11        for(int i = b; i < 0; sum--, i++);
12    return sum;
13 }
```

funções IX

Para chamar uma função, passamos os **argumentos** adequados e usamos o **retorno** de forma apropriada (tipo adequado).

Funções podem não receber argumentos. Nesse caso, usamos **void** nos parâmetros para declarar explicitamente que a função não aceita nenhuma entrada:

```
1 int get_size(void);  
2 int add(int a, int b);
```

Em C, **int f()** (parênteses vazios) **não** significa “zero argumentos” — significa “número indeterminado de argumentos”. Usar **void** é a forma correta e precisa.

funções X

Quando usar `void` nos parâmetros? Quando a função obtém seus dados por outros meios (lendo do teclado, acessando variáveis globais etc) ou simplesmente não precisa de entrada.

```
1 int get_size(void) {  
2     int n;  
3     scanf("%d", &n);  
4     return n;  
5 }
```

Da mesma forma, funções podem não retornar valor. Declaramos que o retorno é `void` :

```
1 void print_grid(int n);
```

exemplo I

Declare uma função chamada `valid_triangle` que usa três números reais que representam os comprimentos dos três lados de um triângulo como seus argumentos e retorna `true` ou `false`, dependendo se esses três comprimentos são capazes de formar um triângulo.

Observe as seguintes fatos sobre triângulos:

Um triângulo só pode ter lados com comprimento positivo.

A soma dos comprimentos de quaisquer dois lados do triângulo deve ser maior que o comprimento do terceiro lado.

exemplo II

```
1 bool valid_triangle(float x, float y, float z) {  
2     // all positive sides  
3     if (x <= 0 || y <= 0 || z <= 0) {  
4         return false;  
5     }  
6     // sum of any two sides greater than third  
7     if ((x + y <= z) || (x + z <= y) ||  
8         (y + z <= x)) {  
9         return false;  
10    }  
11    // we're good!  
12    return true;  
13 }
```

níveis de abstração I

meow2.c

```
1 #include <stdio.h>
2
3 void meow(void);
4
5 int main(void) {
6     for (int i = 0; i < 3; i++) {
7         meow();
8     }
9 }
10
11 void meow(void) {
12     printf("meow\n");
13 }
```

níveis de abstração II

Usando parâmetros podemos alterar o nível de abstração de nosso código.

meow3.c

```
1 #include <stdio.h>
2
3 void meow(int n) {
4     for (int i = 0; i < n; i++)
5         printf("meow\n");
6 }
7
8 int main(void) {
9     meow(3);
10 }
```

parâmetros e argumentos I

mario.c

```
1 int get_size(void);  
2 void print_grid(int n);  
3  
4 int main(void) {  
5     int x = get_size();  
6     print_grid(x);  
7 }
```

A variável `x` foi declarada em `main`. Seu valor foi passado como argumento e atribuído ao parâmetro `n` da função `print_grid`.

parâmetros e argumentos II

Em programação, **parâmetro** e **argumento** são termos usados em conjunto para referir-se à passagem de valores para uma função ou método.

Um **parâmetro** é um valor definido na declaração da função, indicando o tipo e nome do valor esperado pela função.

Um **argumento** é o valor real passado para um parâmetro na chamada da função.